UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

Ingeniería de Requisitos

Sílabo y material de estudio

Código de la asignatura: **ISR-401**

Cuarto nivel — 3 créditos — 96 horas presenciales

*Material de estudio elaborado conforme al estándar internacional **IEEE/ISO/IEC 29148:2018**, la guía **SWEBOK v4.0 (2024)**, los modelos de calidad **ISO/IEC 25010:2023** y la segunda edición del manual de Pohl (Springer, 2024).*

Período académico: 2025 – 2026

Quevedo – Los Ríos – Ecuador

«La especificación de requisitos es el momento de la ingeniería del software en el que se decide, en última instancia, si el sistema va a resolver el problema correcto. Todo lo que viene después es construcción; lo que viene antes es ingeniería de requisitos. »

— *adaptado de Klaus Pohl [**Pohl2024**]*

Aviso: Este material acompaña al *Programa Analítico* oficial de la asignatura aprobado por la Facultad de Ciencias de la Computación. Los contenidos están enmarcados en los estándares y guías internacionales que se citan a lo largo del texto y se sustentan en bibliografía verificable, según las normas IEEE.

Índice general

1. Encuadre académico y fundamentos de requisitos de software **1**

1.1. Tema 1. Encuadre académico de la asignatura 1

1.1.1. Datos generales de la asignatura 2

1.1.2. Mapa global del semestre 2

1.1.3. Metodología y estrategias didácticas 2

1.1.4. Sistema de evaluación 3

1.1.5. Introducción a la Ingeniería de Requisitos 3

1.2. Tema 2. Fundamentos de requisitos de software 5

1.2.1. Subtema 2.1. Definición de requisito de software. Requisitos de productos vs. requisitos de procesos 5

1.2.2. Subtema 2.2. Requisitos funcionales y no funcionales. Propiedades emergentes. Requisitos cuantificables 8

1.2.3. Subtema 2.3. Requisitos del sistema vs. requisitos del software. Restricciones y dependencias 11

1.2.4. Subtema 2.4. Taxonomía de requisitos de calidad (ISO/IEC 25010:2023). Los llamados NFR 15

1.3. Mapa síntesis de la Semana 1 20

1.4. Práctica experimental y trabajo autónomo 20

1.5. Resumen ejecutivo de la semana 21

2. Proceso de ingeniería de requisitos y contexto del sistema **23**

2.1. Tema 3. Proceso de requisitos 23

2.1.1. Subtema 3.1. Modelos de proceso de requisitos. Actores 23

2.1.2. Subtema 3.2. Administración y soporte de procesos. Interrelación con el ciclo de vida del software 26

2.1.3. Subtema 3.3. Perfeccionamiento y calidad del proceso de IR 27

2.1.4. Subtema 3.4. El framework de IR: actividades core, actividades transversales y artefactos 29

2.2. Tema 4. Contexto, tipos de sistemas y visión 30

2.2.1. Subtema 4.1. Tipos de sistemas y consecuencias para IR 30

2.2.2. Subtema 4.2. Límite del sistema, descomposición y visión 32

2.2.3. Subtema 4.3. Las ocho perspectivas de contexto de Pohl 34

2.3. Mapa síntesis de la Semana 2 37

2.4. Práctica experimental y trabajo autónomo 38

2.5. Resumen ejecutivo de la semana 39

3. Elicitación de requisitos: técnicas clásicas **41**

3.1. Tema 5. Elicitación de requisitos 41

3.1.1. Subtema 5.1. Fundamentos y fuentes de la elicitación 41

3.1.2. Subtema 5.2. Entrevistas y cuestionarios . . . 43
3.1.3. Subtema 5.3. Talleres de requisitos y grupos focales . . . 45
3.1.4. Subtema 5.4. Observación y etnografía . . . 46
3.1.5. Subtema 5.5. Análisis de documentos y sistemas existentes . . . 48
3.2. Mapa síntesis de la Semana 3 . . . 50
3.3. Práctica experimental y trabajo autónomo . . . 50
3.4. Resumen ejecutivo de la semana . . . 51

4. Técnicas avanzadas de elicitación y modelado de metas 52
4.1. Tema 6. Técnicas avanzadas de elicitación . . . 52
4.1.1. Subtema 6.1. Prototipado como técnica de elicitación . . . 52
4.1.2. Subtema 6.2. Modelado orientado a metas: i* y KAOS . . . 54
4.1.3. Subtema 6.3. Modelado de procesos de negocio con BPMN . . . 56
4.1.4. Subtema 6.4. Historias de usuario y técnicas ágiles de elicitación . . . 57
4.2. Mapa síntesis de la Semana 4 . . . 60
4.3. Práctica experimental y trabajo autónomo . . . 60
4.4. Resumen ejecutivo de la semana . . . 61

5. Análisis de requisitos: conflictos, priorización y calidad 62
5.1. Tema 7. Análisis de requisitos . . . 62
5.1.1. Subtema 7.1. Análisis y clasificación de requisitos . . . 62
5.1.2. Subtema 7.2. Detección y resolución de conflictos entre requisitos . . . 64
5.1.3. Subtema 7.3. Técnicas de priorización de requisitos . . . 65
5.1.4. Subtema 7.4. Calidad de los requisitos y trazabilidad . . . 67
5.2. Mapa síntesis de la Semana 5 . . . 70
5.3. Práctica experimental y trabajo autónomo . . . 70
5.4. Resumen ejecutivo de la semana . . . 71

6. Metas, escenarios y requisitos orientados a solución 72
6.1. Tema 7. Metas, escenarios y requisitos orientados a solución . . . 72
6.1.1. Subtema 7.1. Metas: concepto, descomposición y documentación . . . 72
6.1.2. Subtema 7.2. Escenarios: tipos, documentación y once reglas . . . 75
6.1.3. Subtema 7.3. Diagramas de CU UML y Message Sequence Charts . . . 79
6.1.4. Subtema 7.4. Requisitos orientados a solución: los tres modelos . . . 81
6.2. Mapa síntesis de la Semana 6 . . . 83
6.3. Práctica experimental y trabajo autónomo . . . 83
6.4. Resumen ejecutivo de la semana . . . 84

7. Documentación de requisitos 85
7.1. Tema 8. Documentación de requisitos . . . 85
7.1.1. Subtema 8.1. Propósito de la documentación y tipos de artefactos . . . 85
7.1.2. Subtema 8.2. Formatos de representación y técnicas de precisión lingüística 86
7.1.3. Subtema 8.3. Atributos de requisito: la ficha de especificación individual 89

7.1.4. Subtema 8.4. Criterios de calidad y criterios de aceptación 91
7.2. Mapa síntesis de la Semana 7 93
7.3. Práctica experimental y trabajo autónomo 93
7.4. Resumen ejecutivo de la semana 94

8. El documento ERS/SRS **95**
8.1. Tema 9. El documento ERS/SRS 95
8.1.1. Subtema 9.1. Tipos de especificaciones en la familia IEEE 29148:2018 . 95
8.1.2. Subtema 9.2. Estructura de referencia del ERS según IEEE 29148:2018 97
8.1.3. Subtema 9.3. Lenguaje natural: verbos modales y técnicas de precisión . 100
8.1.4. Subtema 9.4. Combinación de modelos UML y texto en el ERS 101
8.2. Mapa síntesis de la Semana 8 104
8.3. Práctica experimental y trabajo autónomo 105
8.4. Resumen ejecutivo de la semana 105

9. Modelos de especificación UML: datos y función **107**
9.1. Subtema 10.1. Modelos de datos: ER y clases UML 107
9.1.1. El modelo Entidad-Relación como puerta de entrada a los datos 107
9.2. Subtema 10.2. Modelos funcionales: DFD y actividades UML 110
9.2.1. El diagrama de flujo de datos en IR 111
9.2.2. Diagramas de actividades UML 113
9.3. Mapa síntesis de la Semana 9 115
9.4. Práctica experimental y trabajo autónomo 115
9.5. Resumen ejecutivo de la semana 116

10. Modelos de comportamiento e interrelaciones **117**
10.1. Subtema 10.3. Autómatas finitos, statecharts y máquinas de estado UML . . . 117
10.1.1. De los autómatas finitos a los statecharts 117
10.2. Subtema 10.4. Interrelaciones entre las tres perspectivas 121
10.2.1. El principio de coherencia de Pohl 121
10.2.2. Checklist de coherencia del ERS (vista de modelos) 122
10.2.3. Integración de los cuatro modelos en el ERS del PFC 123
10.3. Mapa síntesis de la Semana 10 123
10.4. Práctica experimental y trabajo autónomo 124
10.5. Resumen ejecutivo de la semana 124

11. Validación de requisitos **126**
11.1. Subtema 11.1. Motivación y principios de validación 126
11.1.1. ¿Por qué validar los requisitos? 126
11.1.2. Los seis principios de validación de Pohl 127
11.2. Subtema 11.2. Walkthroughs, revisiones estructuradas e inspecciones 127
11.2.1. Las tres modalidades de revisión de requisitos 128
11.3. Subtema 11.3. Prototipado como técnica de validación 130

11.3.1. El prototipo de validación vs. el prototipo de elicitación 130
11.3.2. Validación de modelos UML 131
11.4. Subtema 11.4. Pruebas de aceptación y relación con requisitos 131
11.4.1. Tipos de pruebas de aceptación 132
11.4.2. Vínculo entre GWT, RTM y plan de pruebas 133
11.5. Mapa síntesis de la Semana 11 134
11.6. Práctica experimental y trabajo autónomo 134
11.7. Resumen ejecutivo de la semana 135

12. Gestión de requisitos **136**
12.1. Subtema 12.1. Naturaleza iterativa de la IR y gestión del contexto 136
12.1.1. La IR no termina cuando empieza el desarrollo 136
12.1.2. Ciclo de vida iterativo de la IR 137
12.1.3. Gestión del contexto: fuentes de cambio en PFC agropecuarios 137
12.2. Subtema 12.2. Gestión del cambio: CCB, proceso y documentación 138
12.2.1. El proceso de gestión del cambio 138
12.3. Subtema 12.3. Seguimiento y trazabilidad de requisitos 139
12.3.1. Los tres tipos de trazabilidad 139
12.3.2. Trazabilidad dirigida por uso 140
12.4. Subtema 12.4. Medición, priorización, configuración y baselines 140
12.4.1. Métricas de gestión de requisitos 141
12.4.2. Gestión de la configuración del ERS 141
12.4.3. Baselines del ERS en el PFC 143
12.5. Mapa síntesis de la Semana 12 143
12.6. Práctica experimental y trabajo autónomo 144
12.7. Resumen ejecutivo de la semana 144

13. Herramientas CASE y estándares para IR **146**
13.1. Subtema 13.1. Herramientas CASE para IR 146
13.1.1. Panorama de herramientas 147
13.2. Subtema 13.2. IEEE 29148:2018 e ISO/IEC 25010:2023 en proyectos reales . . 148
13.2.1. IEEE 29148:2018: más allá del ERS 148
13.2.2. ISO/IEC 25010:2023 como criterio de RNF 149
13.3. Subtema 13.3. Atributos y trazabilidad en herramientas CASE 150
13.3.1. Esquema de atributos configurado en Jira para el PFC 150
13.4. Subtema 13.4. Gestión de la configuración del ERS en Git 151
13.4.1. Flujo de trabajo Git para el ERS del PFC 151
13.5. Mapa síntesis de la Semana 13 152
13.6. Práctica experimental y trabajo autónomo 153
13.7. Resumen ejecutivo de la semana 153

14. Ingeniería de Requisitos en procesos ágiles **154**

14.1. Subtema 14.1. IR en procesos ágiles: backlog, historias y criterios 154
14.1.1. El product backlog como ERS ágil 154
14.2. Subtema 14.2. BDD: especificación ejecutable de requisitos 156
14.2.1. La anatomía completa de un escenario Gherkin 156
14.2.2. De Gherkin a prueba ejecutable con Pytest-BDD 158
14.3. Subtema 14.3. Integración de IR con Scrum 158
14.3.1. El rol del Product Owner en la IR ágil 158
14.3.2. IR en cada evento Scrum 159
14.4. Subtema 14.4. XP y comparación IR tradicional vs. IR ágil 159
14.4.1. IR en Extreme Programming (XP) 159
14.4.2. Comparación IR tradicional vs. IR ágil 160
14.5. Mapa síntesis de la Semana 14 161
14.6. Práctica experimental y trabajo autónomo 161
14.7. Resumen ejecutivo de la semana 162

15. Proyecto integrador: planificación, levantamiento y análisis 163
15.1. Subtema 15.1. Selección del sistema, stakeholders y plan de IR 163
15.1.1. Criterios para seleccionar el sistema objetivo del PFC 163
15.1.2. Mapa de stakeholders del PFC 164
15.1.3. Plan de IR del PFC 165
15.2. Subtema 15.2. Elicitación con usuarios reales 166
15.2.1. Guía de entrevista real para el PFC agropecuario 166
15.2.2. Taller de requisitos real: estructura para el PFC 166
15.3. Subtema 15.3. Clasificación, priorización y negociación 167
15.3.1. Clasificación de los candidatos a requisitos del PFC 167
15.3.2. Negociación de conflictos reales del PFC 168
15.4. Mapa síntesis de la Semana 15 168
15.5. Actividades del proyecto integrador 169
15.6. Resumen ejecutivo de la semana 169

16. Especificación, modelado, validación y defensa del PFC 170
16.1. Subtema 16.1. ERS/SRS completo según IEEE 29148 170
16.1.1. Lista de verificación del ERS del PFC 170
16.2. Subtema 16.2. Modelos UML: revisión final y coherencia 171
16.2.1. Revisión de coherencia de los modelos del PFC 171
16.3. Subtema 16.3. Revisión cruzada entre equipos 172
16.3.1. Protocolo de peer review entre PFC 172
16.4. Subtema 16.4. Métricas, trazabilidad en CASE y rubrica de evaluación 173
16.4.1. Métricas de calidad del ERS del PFC con ISO 25010 173
16.5. Subtema 17.1–17.3. Defensa final del PFC 174
16.5.1. Estructura de la presentación oral del ERS 174
16.5.2. Simulacro de solicitudes de cambio en la defensa 175

16.6. Mapa síntesis de la Semana 16 y cierre del curso . . . 175
16.7. Actividades finales del proyecto integrador . . . 176
16.8. Resumen ejecutivo de la semana y del curso . . . 176

17. Métricas, trazabilidad y preparación de la defensa 177
17.1. Subtema 17.1. Trazabilidad en CASE y configuración final del ERS . . . 177
17.1.1. Verificación final de la trazabilidad en la herramienta CASE . . . 177
17.1.2. Estado final del repositorio Git del PFC . . . 179
17.2. Subtema 17.2. Evaluación del ERS con rúbrica: autoevaluación final . . . 179
17.2.1. Autoevaluación del ERS con la rúbrica institucional . . . 179
17.2.2. Errores más frecuentes detectados en defensas anteriores . . . 180
17.2.3. Completitud, consistencia y verificabilidad: checklist final . . . 181
17.3. Subtema 17.3. Preparación y estructura de la defensa final . . . 182
17.3.1. Estructura de la presentación: los 20 minutos más importantes . . . 183
17.3.2. Simulacro de respuesta a solicitudes de cambio . . . 184
17.4. Mapa síntesis de la Semana 17 . . . 185
17.5. Actividades de la semana 17 . . . 185
17.6. Resumen ejecutivo de la semana . . . 186

18. Defensa formal, evaluación y cierre del semestre 187
18.1. Protocolo de la defensa formal del PFC . . . 187
18.1.1. Estructura de la sesión de defensa . . . 187
18.1.2. Rúbrica del tribunal: cómo se aplica la calificación . . . 188
18.1.3. Preguntas más frecuentes del tribunal . . . 189
18.2. Evaluación final y coevaluación entre equipos . . . 191
18.2.1. Coevaluación: los estudiantes como tribunal . . . 191
18.3. Reflexión final: el proceso de IR visto desde el final del semestre . . . 191
18.3.1. ¿Qué hemos aprendido sobre IR en 18 semanas? . . . 191
18.3.2. Las diez lecciones del semestre . . . 192
18.3.3. El camino hacia la certificación IREB . . . 193
18.4. Cierre del semestre: evaluación del proceso de aprendizaje . . . 194
18.4.1. Autoevaluación y coevaluación del proceso . . . 194
18.4.2. Mensaje de cierre . . . 194
18.5. Mapa de cierre del semestre completo . . . 195

Índice de figuras

1.1. Secuencia de las cinco unidades de la asignatura y su distribución aproximada en el calendario semestral. 2

1.2. Framework de Ingeniería de Requisitos según Pohl: tres clases de artefactos, tres actividades centrales y dos actividades transversales, todo enmarcado por el contexto del sistema. Adaptado de [**Pohl2024**]. 4

1.3. Cadena de transformación de la necesidad al requisito de software. Adaptado de [**IEEE29148_2018**, **Pohl2024**, **SWEBOK_v4_2024**]. 6

1.4. Pirámide invertida de niveles de requisitos. Cada nivel se deriva del anterior mediante un proceso de *refinamiento y asignación* (allocation) [**IEEE29148_2018**]. 12

1.5. Las nueve características de calidad de ISO/IEC 25010:2023 [**ISO25010_2023**]. La novena característica, **Safety**, constituye la novedad principal de la edición 2023. 16

2.1. Tres modelos de proceso de Ingeniería de Requisitos (IR). (a) En cascada: actividades ejecutadas una vez en secuencia. (b) Iterativo: ciclos sucesivos refinan el conjunto de requisitos. (c) Ágil: requisitos descubiertos y desarrollados sprint a sprint. Adaptado de [**Royce1970**, **Boehm1988**, **Sommerville2016**, **Cao2008**]. 24

2.2. La IR en el ciclo de vida del software. Adaptado de [**ISO12207_2017**, **SWEBOK_v4_2024**, **Pohl2024**]. 27

2.3. Framework completo de IR de Pohl 2024: contexto, tres artefactos, tres actividades core y dos actividades transversales [**Pohl2024**]. 29

2.4. Sistema, contexto y entorno: los tres niveles que la IR debe distinguir. Adaptado de [**Pohl2024**]. 33

2.5. Las ocho perspectivas de contexto de Pohl 2024 [**Pohl2024**]. 35

3.1. Ciclo de cuatro pasos de la elicitación de requisitos. La flecha roja indica que la validación suele generar nuevas preguntas y una nueva iteración. Adaptado de [**Pohl2024**, **Robertson2012**]. 43

4.1. Modelo SD simplificado en i* para el sistema de turnos del Ministerio de Salud Pública del Ecuador (MSP). Las dependencias muestran lo que cada actor necesita de los demás. Adaptado de [**Yu2011**, **Pohl2024**]. 55

5.1. Curvas del Modelo Kano. Los requisitos básicos no elevan la satisfacción aunque se implementen bien; los atractivos generan un salto de satisfacción no anticipado. Adaptado de [**Berger1993**, **Wiegers2013**]. 67

6.1. Grafo KAOS del sistema de turnos MSP. OR-refinement en nivel 1 (alternativas de diseño); AND-refinement en nivel 2 (todas las sub-metas son necesarias). Las meta hojas verdes operacionalizan en RF; el óvalo dorado es una suposición sobre el entorno. Adaptado de [**vanLamsweerde2001**, **Pohl2024**]. 74

6.2. MSC del CU-01: Reservar turno médico (flujo básico completo). Flechas sólidas = mensajes entrantes; flechas discontinuas = respuestas. El rectángulo azul en la lifeline del sistema indica el período de procesamiento. Adaptado de [**Pohl2024**, **Larman2005**]. 80
6.3. Diagrama de clases UML del dominio del sistema de turnos MSP. Se modela el *problema*, no el diseño. Las multiplicidades expresan reglas de negocio verificables. Adaptado de [**Fowler2004**, **Pohl2024**]. 81
6.4. Statechart del ciclo de vida de la entidad `Turno`. Cada transición muestra el evento disparador y la acción ejecutada por el sistema (separados por "/"). Adaptado de [**Harel1987**, **Pohl2024**]. 82

7.1. Continuum de formalidad de los formatos de documentación de requisitos. Las tres flechas muestran los tres atributos que varían a lo largo del eje. Adaptado de [**Pohl2024**, **IEEE29148_2018**]. 86

8.1. Jerarquía de documentos de requisitos de la familia IEEE 29148:2018. Las flechas indican el flujo de derivación. En proyectos pequeños a medianos, el Especificación de Requisitos de Software (ERS)/SRS (verde) puede ser el único documento formal. Adaptado de [**IEEE29148_2018**, **ISO15288_2023**]. . . . 96
8.2. Ciclo de vida del ERS: desde el primer borrador hasta el baseline y la gestión de cambios posterior. El Change Control Board (Comité de Control de Cambios) (CCB) es el árbitro de todos los cambios al baseline. Adaptado de [**IEEE29148_2018**, **Pohl2024**, **Wiegers2013**]. 103

9.1. Diagrama ER del sistema de gestión ganadera (PFC — Los Ríos). Notación simplificada Crow's Foot. El arete es la clave primaria del bovino, conforme al sistema de identificación AGROCALIDAD. Adaptado de [**Chen1976**, **Pohl2024**].109
9.2. Diagrama de clases UML (dominio) del sistema de clínica veterinaria (PFC — Quevedo). La composición entre Consulta y Receta indica que la receta no existe sin una consulta. Adaptado de [**Fowler2004**, **Larman2005**]. 110
9.3. DFD Nivel 0 (diagrama de contexto) del sistema de gestión de camaroneras. Las entidades externas identifican las interfaces del sistema y generan directamente los RF de integración. Adaptado de [**DeMarco1978**, **Yourdon1989**]. 112
9.4. DFD Nivel 1 del sistema de camaroneras. Cada burbuja de proceso genera al menos un RF; cada depósito de datos genera entidades en el ER. Adaptado de [**DeMarco1978**, **Yourdon1989**, **Pohl2024**]. 112
9.5. Diagrama de actividades UML del proceso de cosecha de palma africana. El fork indica ejecución paralela (el cortador descarga mientras el pesador registra). La decisión modela la regla de negocio del peso mínimo de cierre de jornada. Cada acción genera RF; el fork genera un RNF de concurrencia. Adaptado de [**OMG_UML_2017**, **Fowler2004**]. 114

10.1. Autómata finito determinista del nodo sensor IoT de la piscina camaronera. Cada transición genera un RF de comportamiento: el evento es el disparador; la acción tras `/` es lo que el sistema ejecuta. Adaptado de [**Pohl2024**, **IEEE29148_2018**].118

10.2. Statechart (Harel) del ciclo de vida del bovino en el PFC de ganadería. El estado compuesto *Etapa Reproductiva* agrupa los sub-estados reproductivos, evitando repetir transiciones. Las guardas (entre corchetes) generan reglas de negocio; las acciones (tras /) generan RF. Adaptado de [**Harel1987**, **Pohl2024**]. 120

10.3. Diagrama de máquina de estados UML para la entidad `Cita` en el sistema de clínica veterinaria. Los estados ATENDIDA, NO ASISTIDA y CANCELADA son estados finales distintos, necesarios para las estadísticas de gestión de la clínica. Adaptado de [**OMG_UML_2017**, **Harel1987**]. 121

11.1. Cadena de trazabilidad desde el requisito hasta el resultado de la prueba de aceptación, pasando por la RTM. Adaptado de [**IEEE29148_2018**, **Adzic2011**].133

12.1. Ciclo de vida iterativo de la IR. La gestión del cambio (centro) actúa como regulador de todas las iteraciones posteriores al primer baseline. Adaptado de [**Pohl2024**, **IEEE29148_2018**]. 137

13.1. Flujo de trabajo Git recomendado para el ERS del PFC. La rama `main` contiene solo versiones aprobadas por el CCB; los cambios entran via Pull Request revisado. Los tags marcan los baselines. Adaptado de [**IEEE29148_2018**, **Pohl2024**]. . 152

15.1. Mapa de stakeholders del PFC de gestión ganadera. Los stakeholders en el cuadrante superior derecho (involucrar activamente) deben participar en todas las sesiones de elicitación y revisión del ERS. Adaptado de [**Robertson2012**, **Pohl2024**]. 165

Índice de tablas

1.1. Datos generales de la asignatura Ingeniería de Requisitos. 2
1.2. Componentes del sistema de evaluación de la asignatura. 3
1.3. Las nueve características de un requisito bien formado, con ejemplos contextualizados. Adaptado de [**IEEE29148_2018**, **Pohl2024**]. 6
1.4. Requisitos de producto vs. requisitos de proceso. 8
1.5. Las tres familias de requisitos no funcionales según [**Sommerville2016**]. . . . 9
1.6. Conversión de enunciados vagos en requisitos cuantificables. 11
1.7. Niveles canónicos de requisitos según IEEE 29148:2018 [**IEEE29148_2018**]. . 12
1.8. Asignación (allocation) de requisitos del sistema al software, con ejemplos ecuatorianos. 13
1.9. Tipos comunes de restricciones, con ejemplos en Ecuador. 14
1.10. Tipos de dependencias entre requisitos. 15
1.11. Cambios principales entre ISO/IEC 25010:2011 e ISO/IEC 25010:2023 [**ISO25010_2011**, **ISO25010_2023**]. 16
1.12. Las nueve características de calidad ISO/IEC 25010:2023 con sus subcaracterísticas [**ISO25010_2023**]. 17
1.13. Mapa de cierre de la Semana 1. 20

2.1. Comparación de los tres modelos de proceso de IR según varios autores [**Sommerville2016**, **Pressman2015**, **Pohl2024**, **Inayat2015**, **Curcio2018**]. 25
2.2. Roles típicos en el proceso de IR. 26
2.3. Modelo Requirements Engineering Process Maturity Model (REPM) de Sawyer, Sommerville y Viller [**Sawyer1997**]. 28
2.4. Tipos de sistemas y sus consecuencias para la IR. 31
2.5. Plantilla de documento de visión adaptada de [**Wiegers2013**]. 34
2.6. Las ocho perspectivas de contexto, sus preguntas guía y ejemplos. 35
2.7. Plantilla de análisis de contexto por perspectivas. 37
2.8. Mapa de cierre de la Semana 2. 38

3.1. Fuentes de requisitos y técnicas de elicitación asociadas. Adaptado de [**Pohl2024**, **IEEE29148_2018**, **Wiegers2013**]. 42
3.2. Tipos de entrevista y sus características [**Wiegers2013**, **Robertson2012**, **Pohl2024**]. 43
3.3. Técnicas de formulación de preguntas para entrevistas de elicitación [**Wiegers2013**, **Robertson2012**, **Pohl2024**]. 44
3.4. Estructura de un taller de requisitos de un día. Adaptado de [**Robertson2012**, **Wiegers2013**]. 45
3.5. Modalidades de observación en IR [**Pohl2024**, **Millen2000**, **Sharp1999**]. . . . 47
3.6. Tipos de documentos útiles para la elicitación en proyectos ecuatorianos. 48

3.7. Comparación de las técnicas clásicas de elicitación por criterios relevantes [**Pohl2024**, **Wiegers2013**, **Robertson2012**, **Zowghi2005**]. 49
3.8. Mapa de cierre de la Semana 3. 50

4.1. Taxonomía de prototipos en IR por fidelidad y técnica. Adaptado de [**Pohl2024**, **Wiegers2013**]. 53
4.2. Operadores de descomposición de metas en KAOS [**vanLamsweerde2001**]. . . 56
4.3. Elementos BPMN 2.0 más relevantes para la elicitación de requisitos [**OMG_BPMN_2013**]. 57
4.4. Criterios INVEST para evaluar la calidad de una historia de usuario [**Cohn2004**]. 58
4.5. Jerarquía de artefactos de requisitos en metodologías ágiles. Adaptado de [**Cohn2004**, **Schwaber2020**]. 59
4.6. Mapa de cierre de la Semana 4. 60

5.1. Ejemplo de clasificación multidimensional de requisitos del sistema de turnos MSP [**Pohl2024**, **IEEE29148_2018**]. 63
5.2. Las cinco categorías de defectos más frecuentes en los requisitos, con ejemplos y correcciones [**Wiegers2013**, **Berry2004**, **Femmer2017**]. 64
5.3. Categorías Must-have, Should-have, Could-have, Won't-have (técnica de priorización) (MoSCoW) y su interpretación [**Wiegers2013**]. 66
5.4. Categorías del Modelo Kano y su impacto en la satisfacción del cliente. Adaptado de [**Berger1993**, **Wiegers2013**]. 66
5.5. Ejemplo de priorización por valor/costo/riesgo para el sistema de matrícula Universidad Técnica Estatal de Quevedo (UTEQ). Ponderaciones: $w_V = 1, w_P = 1, w_C = 1, w_R = 0{,}5$. 67
5.6. Características de calidad del conjunto de requisitos según [**IEEE29148_2018**]. 68
5.7. Ejemplo de Requirements Traceability Matrix (Matriz de Trazabilidad de Requisitos) (RTM) para el sistema de matrícula UTEQ. 69
5.8. Mapa de cierre de la Semana 5. 70

6.1. Taxonomía de metas según [**vanLamsweerde2001**, **Pohl2024**, **IEEE29148_2018**]. 73
6.2. Las siete reglas de Pohl para la documentación textual de metas [**Pohl2024**]. Se incluye contraejemplo y corrección para cada regla. 74
6.3. Elementos del modelo SR en iStar 2.0 [**iStar20_2016**, **Yu2011**]. 75
6.4. Tipos de escenario y sus características [**Pohl2024**, **Robertson2012**, **Cockburn2001**, **Sindre2005**, **Cohn2004**]. 76
6.5. CU-01: Reservar turno médico — plantilla Cockburn *fully dressed* [**Cockburn2001**, **Pohl2024**]. 77
6.6. MC-01: Suplantación de identidad para reservar turno masivamente. Adaptado de [**Sindre2005**, **Pohl2024**, **Ecuador_LOPDP_2021**]. 79
6.7. Relaciones en los diagramas de CU UML [**Larman2005**, **Pohl2024**, **Cockburn2001**]. 80
6.8. Las tres perspectivas de los modelos orientados a solución [**Pohl2024**, **Fowler2004**, **Harel1987**]. 81
6.9. Tabla de coherencia entre los tres modelos ROS del sistema de turnos MSP. Cada fila verifica que una entidad o evento aparezca consistentemente en los tres modelos [**Pohl2024**, **IEEE29148_2018**]. 82

6.10. Mapa de cierre de la Semana 6. 83

7.1. Tipos de artefactos de documentación de requisitos [**Pohl2024**, **IEEE29148_2018**]. 86
7.2. Los ocho *requirement smells* más frecuentes según Femmer et al. [**Femmer2017**] y Tiwari et al. [**Tiwari2022**]. 88
7.3. Los cinco patrones de la plantilla EARS con ejemplos ecuatorianos [**Mavin2009**, **Pohl2024**]. 89
7.4. Los diez atributos de un requisito individual según [**IEEE29148_2018**, **Robertson2012**, **Wiegers2013**]. 90
7.5. Ficha Volere para RF-01: Reservar turno médico. Adaptado de [**Robertson2012**]. 91
7.6. Criterios de aceptación en formato GWT para RF-01 y RNF-03. Adaptado de [**Adzic2011**, **IEEE29148_2018**]. 92
7.7. Métricas de calidad del proceso de IR más usadas en proyectos medianos [**Pohl2024**, **Wiegers2013**]. 92
7.8. Mapa de cierre de la Semana 7. 93

8.1. Familia de documentos IEEE/ISO/IEC 29148:2018 y su posición en la jerarquía de requisitos [**IEEE29148_2018**, **ISO15288_2023**]. 96
8.2. Criterios de selección del tipo de especificación según el contexto del proyecto [**IEEE29148_2018**, **Pohl2024**]. 97
8.3. Estructura de referencia canónica del ERS/SRS según [**IEEE29148_2018**, **Wiegers2013**, **Robertson2012**]. 97
8.4. Capítulo 4 del ERS del sistema de matrículas UTEQ. Ejemplo de organización de RF por subsistema funcional. 99
8.5. Jerarquía de verbos modales en la redacción de requisitos según [**IEEE29148_2018**, **Pohl2024**, **Rupp2009**]. 100
8.6. Estrategias de integración de modelos UML y texto en el ERS [**Pohl2024**, **Fowler2004**, **Larman2005**]. 102
8.7. RTM extendida para los RF más críticos del sistema de matrícula UTEQ. La trazabilidad es bidireccional en todas las columnas [**IEEE29148_2018**, **ISO15288_2023**]. 103
8.8. Proceso mínimo del CCB para proyectos universitarios y medianos [**IEEE29148_2018**, **Wiegers2013**]. 104
8.9. Mapa de cierre de la Semana 8. 104

9.1. Comparación de las dos notaciones ER más usadas en IR [**Chen1976**, **Fowler2004**].108
9.2. Reglas de mapeo de ER a diagrama de clases UML [**Fowler2004**, **Larman2005**, **OMG_UML_2017**]. 109
9.3. Niveles del DFD y su uso en la especificación de requisitos [**DeMarco1978**, **Yourdon1989**, **SWEBOK_v4_2024**]. 111
9.4. Mapa de cierre de la Semana 9. 115

10.1. Tabla de transiciones del AFD del sensor IoT y los RF derivados. 119

10.2. Tabla de interrelaciones entre los tres modelos del PFC de ganadería. Cada celda vacía indica una posible omisión que el analista debe justificar o corregir [**Pohl2024**, **IEEE29148_2018**, **ISO15288_2023**]. 122
10.3. Checklist de coherencia de los tres modelos del ERS [**Pohl2024**, **IEEE29148_2018**, **Wiegers2013**]. 122
10.4. Mapa de cierre de la Semana 10. 123

11.1. Costo relativo de corrección de defectos de requisitos según la fase de detección [**Boehm2001**, **Hofmann2001**, **StandishGroup2020**]. 127
11.2. Comparación de las tres modalidades de revisión de requisitos [**Wiegers2013**, **Pohl2024**, **Fagan1976**, **Robertson2012**]. 128
11.3. Las cuatro fases de la inspección Fagan aplicada al ERS del PFC de camaroneras [**Fagan1976**, **Wiegers2013**]. 129
11.4. Comparación entre prototipo de elicitación y de validación [**Pohl2024**, **Wiegers2013**, **Floyd1984**]. 130
11.5. Tipos de pruebas de aceptación y su relación con los artefactos del ERS [**IEEE29148_2018**, **Adzic2011**, **Wiegers2013**]. 132
11.6. Fragmento del plan de pruebas de aceptación del PFC de aulas IoT. Cada caso de prueba está vinculado a un requisito del ERS con trazabilidad directa [**IEEE29148_2018**, **Adzic2011**]. 133
11.7. Mapa de cierre de la Semana 11. 134

12.1. Formulario de solicitud de cambio (CR) del PFC de clínica veterinaria. Este es el formato mínimo recomendado [**Wiegers2013**, **IEEE29148_2018**]. 138
12.2. Proceso del CCB adaptado para proyectos de fin de carrera (PFC) de la UTEQ [**Wiegers2013**, **Pohl2024**]. 139
12.3. Los tres tipos de trazabilidad de requisitos según [**Pohl2024**, **IEEE29148_2018**, **ISO15288_2023**]. 139
12.4. RTM completa con los tres tipos de trazabilidad para el PFC de mascotas a domicilio. Cada fila es un RF trazado en las seis dimensiones [**Pohl2024**, **IEEE29148_2018**, **ISO15288_2023**]. 140
12.5. Métricas clave de gestión de requisitos y sus valores de referencia [**Pohl2024**, **Wiegers2013**, **Wohlin2012**]. 141
12.6. Política de versionado semántico para el ERS del PFC [**IEEE29148_2018**, **Pohl2024**, **Wiegers2013**]. 142
12.7. Los tres baselines del ERS recomendados para un PFC de la UTEQ [**IEEE29148_2018**, **Pohl2024**, **Wiegers2013**]. 143
12.8. Mapa de cierre de la Semana 12 y de la Unidad 4. 143

13.1. Comparación de herramientas CASE para IR relevantes para PFC de la UTEQ [**Wiegers2013**, **Pohl2024**, **SWEBOK_v4_2024**]. 147
13.2. Recomendación de herramienta CASE según el tipo de PFC de la UTEQ. . . . 148
13.3. Cláusulas de IEEE 29148:2018 y su implementación práctica en el PFC de aulas IoT [**IEEE29148_2018**, **Pohl2024**]. 149

13.4. Aplicación de las características de ISO 25010:2023 al PFC de gestión de ganado bovino [**ISO25010_2023**, **Pohl2024**, **Glinz2007**]. 150
13.5. Mapeo de los 10 atributos IEEE 29148:2018 a campos de Jira. Este esquema es el mínimo exigible en el PFC de la UTEQ. 151
13.6. Mapa de cierre de la Semana 13. 152

14.1. Jerarquía de artefactos del backlog ágil y su equivalente en IR tradicional [**Cohn2004**, **Schwaber2020**, **Pohl2024**]. 155
14.2. Palabras clave de Gherkin y su semántica [**North2006**, **Adzic2011**]. 156
14.3. Actividades de IR en cada evento de Scrum [**Schwaber2020**, **Pohl2024**, **IREB_CPRE_2024**]. 159
14.4. Comparación sistemática entre IR tradicional e IR ágil [**Pohl2024**, **Cohn2004**, **Schwaber2020**, **IREB_CPRE_2024**]. 160
14.5. Mapa de cierre de la Semana 14. 161

15.1. Criterios de selección del sistema objetivo para el PFC de la UTEQ con ejemplos de los seis dominios propuestos. 164
15.2. Plan de IR mínimo para el PFC de la UTEQ (adaptable a los 6 dominios propuestos) [**IEEE29148_2018**, **Pohl2024**, **Robertson2012**]. 165
15.3. Agenda del taller de requisitos del PFC (adaptación de la estructura de Robertson & Robertson [**Robertson2012**]). 167
15.4. Mapa de cierre de la Semana 15 — inicio del Proyecto Integrador. 168

16.1. Lista de verificación del ERS del PFC — criterios de completitud. Todos los ítems deben estar marcados como ✓antes de la entrega. 171
16.2. Elementos UML críticos a revisar por dominio de PFC. 172
16.3. Rúbrica de evaluación del ERS del PFC — Ingeniería de Requisitos UTEQ. Puntaje total: 100 puntos. 174
16.4. Estructura recomendada de la presentación oral del ERS del PFC. 174
16.5. Mapa de cierre del curso ISR-401 Ingeniería de Requisitos. 175

17.1. Verificación de la cobertura de trazabilidad por tipo — PFC semana 17. Toda celda en rojo indica una brecha que debe cerrarse antes de la defensa [**IEEE29148_2018**, **Pohl2024**, **ISO15288_2023**]. 178
17.2. Rúbrica de autoevaluación del ERS del PFC — semana 17. El equipo completa las columnas de puntaje propio y evidencia antes de la defensa. 180
17.3. Los diez errores más frecuentes detectados por tribunales en defensas de ERS de PFC, organizados por frecuencia. 181
17.4. Verificación de completitud, consistencia y verificabilidad del ERS del PFC — checklist final de la semana 17 [**Pohl2024**, **IEEE29148_2018**, **Wiegers2013**]. 182
17.5. Guión detallado de la presentación oral del ERS del PFC por minuto. La columna "Evidencia" indica el artefacto que debe estar visible en la pantalla durante ese segmento. 183
17.6. Mapa de cierre de la Semana 17 — semana de verificación y ensayo. 185

18.1. Protocolo completo de la sesión de defensa del PFC de IR — Semana 18. . . . 188

18.2. Niveles de logro de la rúbrica del tribunal para la defensa del ERS del PFC. . . 189
18.3. Las doce preguntas más frecuentes del tribunal en defensas de ERS de PFC, organizadas por tema. 190
18.4. Competencias de IR adquiridas en el semestre y su valoración en el mercado laboral del sector tecnológico ecuatoriano. 192
18.5. Correspondencia entre los contenidos del semestre ISR-401 y los bloques de conocimiento del IREB CPRE v3.2.0 [**IREB_CPRE_2024**, **Pohl2024**]. . . . 193
18.6. Mapa completo del semestre ISR-401 — de la Semana 1 a la Semana 18. 195

Semana 1

Encuadre académico y fundamentos de requisitos de software

Los requisitos no son lo que el cliente dice que quiere; son lo que el sistema debe hacer para resolver, de forma verificable y acordada, un problema real bien delimitado.
adaptado de **Pohl2024**

Objetivos de aprendizaje de la semana

Al finalizar esta primera semana, el estudiante estará en capacidad de:

1. Comprender el alcance, la metodología y el sistema de evaluación de la asignatura **Ingeniería de Requisitos (ISR-401)** y su articulación con el proyecto integrador del semestre.
2. Definir con precisión qué es un **requisito de software** a la luz de la norma **IEEE/ISO/IEC 29148:2018** y de la guía **SWEBOK v4.0** [**IEEE29148_2018**, **SWEBOK_v4_2024**].
3. Distinguir entre **requisitos de producto** y **requisitos de proceso**, y aplicar la diferencia a casos reales del entorno productivo ecuatoriano.
4. Diferenciar **requisitos funcionales** de **no funcionales**, identificar **propiedades emergentes** y redactar requisitos **cuantificables** y verificables.
5. Distinguir entre **requisitos del sistema** y **requisitos del software**, y modelar las **restricciones** y **dependencias** que los condicionan.
6. Aplicar el modelo de calidad **ISO/IEC 25010:2023** para clasificar y especificar requisitos no funcionales con base en sus nueve características de calidad [**ISO25010_2023**].

1.1 Tema 1. Encuadre académico de la asignatura

La asignatura **Ingeniería de Requisitos** ocupa un lugar singular en el plan de estudios de la Carrera de Ingeniería de Software de la UTEQ: es la primera materia profesionalizante en la que el estudiante no construye software sino que decide qué software se va a construir. Esta diferencia, aparentemente sutil, tiene consecuencias profundas en la formación: exige desarrollar competencias comunicativas, analíticas y éticas que no se ejercitan con la misma intensidad en las asignaturas de programación [**UTEQ_PlanIR_2025**, **Daun2023**].

Diversos estudios han mostrado que entre el 40 % y el 60 % de los defectos encontrados en sistemas en producción tienen su origen en una deficiente IR; corregir un defecto en producción cuesta hasta cien veces más que detectarlo en la fase de requisitos [**Boehm2001**, **Charette2005**, **StandishGroup2020**]. La consecuencia es directa: un proyecto de software que descuide los requisitos tiende a fracasar, mientras que un equipo que domine la IR incrementa de manera notable la probabilidad de éxito y reduce el costo total del ciclo de vida

[**Wiegers2013**, **Pohl2024**].

1.1.1 Datos generales de la asignatura

La tabla 1.1 sintetiza la información administrativa oficial de la asignatura conforme al programa analítico de la UTEQ [**UTEQ_PlanIR_2025**].

Tabla 1.1. *Datos generales de la asignatura Ingeniería de Requisitos.*

Atributo	Detalle
Carrera	Ingeniería de Software (rediseño)
Nivel	4to nivel
Modalidad	Presencial
Código	ISR-401
Créditos	3.0
Horas presenciales	96.0 horas teórico-prácticas
Aprendizaje autónomo	48.0 horas
Prerrequisito	Fundamentos de Ingeniería de Software (3er nivel)
Eje de formación	Formación profesional
Cátedra integradora	Sí; integra Calidad, Seguridad y Deontología

1.1.2 Mapa global del semestre

El semestre se organiza en **cinco unidades** que evolucionan desde la teoría hacia la práctica integradora. La figura 1.1 representa esta secuencia.

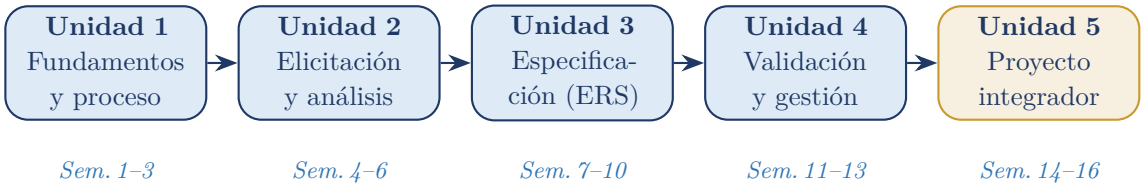


Figura 1.1. *Secuencia de las cinco unidades de la asignatura y su distribución aproximada en el calendario semestral.*

La quinta unidad, **Proyecto integrador**, no introduce nueva teoría: recoge y aplica todo lo aprendido en las cuatro primeras unidades a un caso real de la región (un Gobierno Autónomo Descentralizado (GAD) cantonal, una cooperativa de ahorro, una clínica privada, un sistema agroindustrial bananero, etc.) y culmina con la defensa pública del ERS ante un tribunal docente [**UTEQ_PlanIR_2025**].

1.1.3 Metodología y estrategias didácticas

La metodología combina exposición teórica activa con un fuerte componente práctico en equipo. Se articulan ocho estrategias coordinadas [**Daun2023**, **UTEQ_PlanIR_2025**]:

- **Clase magistral activa**: exposición de fundamentos teóricos (IEEE 29148, SWEBOK v4, ISO/IEC 25010) con preguntas y análisis de casos en tiempo real.
- **Aprendizaje Basado en Problemas (ABP)**: análisis de proyectos reales fallidos por mala gestión de requisitos –por ejemplo, sistemas gubernamentales y de salud públicos del Ecuador y de la región– en los que el estudiante diagnostica la causa raíz desde el ángulo de la IR.
- **Aprendizaje Basado en Proyectos (ABProy)**: desarrollo durante el semestre de un ERS completo de un sistema real, en equipos de 3 a 5 estudiantes, con roles definidos.
- **Aprendizaje colaborativo**: revisión cruzada (peer review) de documentos ERS entre equipos.
- **Estudios de casos**: análisis de pliegos del Servicio Nacional de Contratación Pública (SERCOP), proyectos de banca y de salud pública con requisitos mal especificados.
- **Design Thinking**: aplicado en la elicitación para identificar necesidades latentes mediante empatización y prototipado rápido.
- **Aula invertida**: lectura previa de fragmentos de IEEE 29148 y SWEBOK v4 para que el aula se dedique a discusión y modelado.
- **Gamificación**: dinámicas de puntos en priorización (MoSCoW, Modelo Kano) y detección de defectos en ERS.

1.1.4 Sistema de evaluación

La evaluación es continua, formativa y sumativa. La distribución típica del puntaje –ajustable por el docente al inicio de semestre– se muestra en la tabla 1.2.

Tabla 1.2. *Componentes del sistema de evaluación de la asignatura.*

Componente	Tipo	Peso	Cuándo
Actividades en clase y autónomas	Formativa	20 %	Todo el semestre
1ª evaluación parcial	Sumativa	20 %	Cierre Unidad 2
2ª evaluación parcial	Sumativa	20 %	Cierre Unidad 4
Proyecto integrador (artefactos)	Sumativa	25 %	Entregas iterativas
Defensa final ante tribunal	Sumativa	15 %	Última semana

1.1.5 Introducción a la Ingeniería de Requisitos

¿Qué es la Ingeniería de Requisitos?

La IR es la disciplina que se ocupa de **descubrir, documentar, acordar y gestionar** lo que un sistema de software debe hacer y bajo qué condiciones debe hacerlo [**Pohl2024**, **SWEBOK_v4_2024**, **Sommerville2016**]. La guía SWEBOK v4 lo expresa con claridad: la IR es la rama de la Ingeniería de Software (IS) dedicada al desarrollo, especificación y gestión de los requisitos a lo largo de todo el ciclo de vida del producto [**SWEBOK_v4_2024**]. La norma [**IEEE29148_2018**] la entiende como un conjunto de procesos del ciclo de vida de los

sistemas y del software dedicados a transformar las necesidades de los stakeholders en requisitos de stakeholder, requisitos del sistema y requisitos del software.

Definición

Ingeniería de Requisitos. Conjunto sistemático y disciplinado de actividades dedicadas a *elicitar, documentar, validar, acordar y gestionar* los requisitos de un sistema (frecuentemente intensivo en software), de modo que sirvan como base contractual y técnica para todas las actividades posteriores del ciclo de vida [**Pohl2024**, **IEEE29148_2018**].

Pohl, en su segunda edición de 2024, propone tres **metas centrales** de la IR [**Pohl2024**]:

1. **Conocer los requisitos relevantes y alcanzar consenso entre los stakeholders**, expresar los requisitos sin ambigüedad y documentarlos según los formatos pactados.
2. **Asegurar la calidad del proceso** para que los requisitos documentados cumplan los criterios establecidos.
3. **Gestionar el contexto** en el que la IR tiene lugar y soportar la evolución de los requisitos a lo largo del ciclo de vida.

La figura 1.2 resume el **framework de IR** que Pohl utiliza como hilo conductor del libro y que esta asignatura adoptará como esquema de referencia. Este marco se compone de tres tipos de *actividades core*, dos *actividades transversales* y tres tipos de *artefactos de requisitos*.

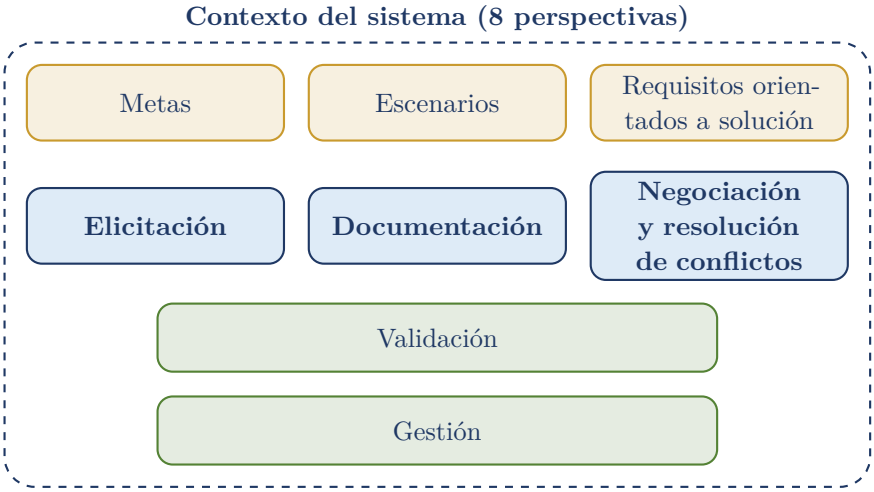


Figura 1.2. *Framework de Ingeniería de Requisitos según Pohl: tres clases de artefactos, tres actividades centrales y dos actividades transversales, todo enmarcado por el contexto del sistema. Adaptado de [***Pohl2024***].*

¿Por qué la IR importa especialmente en Ecuador?

Ecuador atraviesa un proceso sostenido de transformación digital en sectores estratégicos: gobierno electrónico, banca, salud pública, educación superior, agroindustria y comercio. Cada uno de esos sectores ha experimentado proyectos retrasados o cancelados por requisitos mal entendidos. El *Plan Nacional de Gobierno Electrónico* y los lineamientos de soberanía tecnológica exigen procesos formales de IR [**Ecuador_PNGE_2018**, **Ecuador_PNDB_2021**], y la *Ley Orgánica de Protección de Datos Personales* de 2021 incorpora obligaciones explícitas que se traducen, ineludiblemente, en requisitos no funcionales de seguridad y privacidad para todos

los sistemas que traten datos personales [**Ecuador_LOPDP_2021**].

Ejemplo en Ecuador

Caso: matrícula universitaria en la UTEQ. Cuando un estudiante se matricula en línea, intervienen requisitos como: «el sistema deberá permitir matricularse en máximo cinco asignaturas por nivel», «deberá validar que el estudiante haya aprobado los prerrequisitos», «deberá responder en menos de 3 segundos en hora pico», «deberá mantener registros por al menos 10 años por exigencia de la Secretaría Nacional de Educación Superior, Ciencia, Tecnología e Innovación (SENESCYT)». Si cualquiera de estos requisitos no se levanta correctamente, el sistema falla en producción durante el período de matrículas, justo cuando más se necesita.

1.2 Tema 2. Fundamentos de requisitos de software

1.2.1 Subtema 2.1. Definición de requisito de software. Requisitos de productos vs. requisitos de procesos

Definición formal según IEEE/ISO/IEC 29148:2018

La cláusula 3.1.19 de la norma [**IEEE29148_2018**] ofrece una definición **tripartita** que conviene memorizar y comprender en cada una de sus acepciones complementarias. Un requisito es:

1. Un enunciado que traduce o expresa una necesidad y sus **restricciones y condiciones** asociadas.
2. Una **condición o capacidad** que debe poseer un sistema, producto, servicio, resultado o componente para satisfacer un acuerdo, estándar, especificación u otros documentos formalmente impuestos.
3. Una **representación documentada** de una condición o capacidad como en (1) o (2).

Tres ideas atraviesan esta definición y conviene fijarlas:

- Todo requisito **nace de una necesidad**: alguien quiere o requiere algo.
- Esa necesidad debe expresarse como **condición o capacidad observable y verificable**, no como un deseo difuso.
- Debe quedar **documentada**, porque solo lo escrito puede ser revisado, acordado y trazado.

Del problema al requisito: la cadena de evolución

SWEBOK v4 [**SWEBOK_v4_2024**] y Pohl [**Pohl2024**] coinciden en que un requisito atraviesa varias etapas antes de estabilizarse: empieza como una necesidad latente del stakeholder, se convierte en una preocupación expresada, luego en una meta, después en un requisito de stakeholder, y finalmente en un requisito del sistema o del software. La figura 1.3 muestra esta cadena.



Figura 1.3. *Cadena de transformación de la necesidad al requisito de software. Adaptado de [***IEEE29148_2018***,* **Pohl2024***,* **SWEBOK_v4_2024***].*

Anatomía de un buen requisito

La cláusula 5.2.4 de IEEE 29148 establece nueve **características** que todo requisito bien formado debe cumplir individualmente [**IEEE29148_2018**]. La tabla 1.3 las sintetiza con ejemplos en el contexto ecuatoriano.

Tabla 1.3. *Las nueve características de un requisito bien formado, con ejemplos contextualizados. Adaptado de [***IEEE29148_2018***,* **Pohl2024***].*

Característica	Significado	Ejemplo (matrícula)
Necesario	Sin él, los stakeholders no aceptan el sistema.	El sistema deberá registrar la matrícula del estudiante.
Apropiado al nivel	Adecuado al nivel del documento (negocio/sistema/software).	Si es de software: «el sistema deberá validar la cédula del estudiante».
No ambiguo	Una sola interpretación posible.	«en menos de 3 segundos», en lugar de «rápidamente».
Completo	Sin información implícita o supuesta.	Especifica entradas, comportamiento y salidas.
Singular	Un único requisito por enunciado.	Separar «registrar»de «notificar por correo».
Factible	Realizable técnica, legal y económicamente.	No exigir respuesta en 1 ms si la red no lo permite.
Verificable	Comprobable por inspección, análisis, demostración o prueba.	Existe una prueba que demuestre el tiempo de respuesta.
Correcto	Refleja con precisión la necesidad real.	Validado con la responsable de Secretaría Académica.
Conforme	Sigue plantilla, estilo y formato del ERS.	Numeración **RF-NN**, redacción en voz activa.

Una plantilla básica para principiantes, recomendada por [**Mavin2009**, **Pohl2024**], es:

Plantilla básica:

[Sujeto] [Verbo modal: deberá / no deberá] [Acción que el sistema realiza] [Objeto / Condición / Restricción medible].

Cuando los requisitos contienen lógica condicional o eventos, conviene adoptar plantillas más estructuradas, como Easy Approach to Requirements Syntax (EARS) [**Mavin2009**], que se estudiarán a fondo en la Unidad 3.

Advertencia / Error frecuente

Error frecuente del estudiante. Confundir un requisito con una *decisión de diseño* o con una *solución técnica*. Si en lugar de «el sistema deberá autenticar al usuario»se escribe «el sistema deberá usar JWT con expiración de 30 minutos», ya no se está especificando un requisito sino una decisión de implementación. La regla de oro es: *el requisito describe* ***qué****; el diseño describe* ***cómo*** [**Sommerville2016**, **Wiegers2013**].

Requisitos de producto vs. requisitos de proceso

Una distinción fundamental introducida por SWEBOK v4 [**SWEBOK_v4_2024**] y reforzada por Sommerville [**Sommerville2016**] es la que separa los **requisitos de producto** de los **requisitos de proceso**. Confundirlos es causa frecuente de discusiones estériles entre cliente y equipo técnico.

Requisitos de producto. Describen el *sistema de software que se va a construir*: lo que hace, cómo se comporta, las cualidades que debe exhibir y las restricciones que debe respetar. Son los protagonistas del ERS y los que generalmente preocupan más al usuario final.

Ejemplos en el contexto ecuatoriano:

- «El sistema deberá calcular el aporte mensual al Instituto Ecuatoriano de Seguridad Social (IESS) aplicando el porcentaje vigente sobre el sueldo bruto del trabajador.»
- «El sistema deberá emitir comprobantes electrónicos en el formato XML exigido por el Servicio de Rentas Internas (Ecuador) (SRI)»[**SRI_FE_2024**].
- «El sistema deberá soportar al menos 1 500 transacciones concurrentes durante la temporada alta de matrículas.»

Requisitos de proceso. Regulan *cómo se desarrollará el software*, no qué hace el software. Imponen condiciones al equipo de desarrollo, no al producto. Provienen casi siempre del cliente, de la organización contratante o de regulaciones [**SWEBOK_v4_2024**].

Ejemplos típicos en proyectos ecuatorianos:

- «El proveedor deberá usar metodología Scrum con sprints de dos semanas.»
- «El proveedor deberá entregar el código fuente en un repositorio Git institucional bajo licencia GPL.»
- «La documentación deberá entregarse en español, conforme a la plantilla IEEE 29148:2018»[**IEEE29148_20**

Cuadro comparativo. La tabla 1.4 resume las diferencias.

Tabla 1.4. *Requisitos de producto vs. requisitos de proceso.*

Dimensión	Producto	Proceso
¿Qué describe?	El sistema construido (qué hace, cómo se comporta).	El proceso de construcción (cómo se desarrolla).
¿A quién obliga?	Al sistema en operación.	Al equipo de desarrollo o al proveedor.
¿Cuándo se verifica?	En pruebas de aceptación del software.	En auditorías de proceso e hitos contractuales.
Origen típico	Usuarios, clientes, regulaciones del dominio.	Contrato, estándares corporativos, política institucional.
Ejemplo Ecuador	Generación del Anexo Transaccional del SRI.	Uso de GitLab institucional para el versionado.

Ejemplo en Ecuador

Ejemplo integrador: licitación pública del SERCOP. Cuando una entidad pública del Ecuador convoca, vía SERCOP, a una licitación para desarrollar un sistema de gestión documental, los pliegos suelen contener mezcladas estas dos clases de requisitos. Por ejemplo: **(a)** «el sistema deberá firmar electrónicamente los oficios con certificado digital»(producto); **(b)** «el oferente deberá entregar manuales en español, en versión digital y física, y con licencia GPL»(proceso). El analista debe *separarlos explícitamente*, pues se gestionan, verifican y aceptan de manera diferente: el primero con pruebas funcionales, el segundo con auditoría de entregables.

1.2.2 Subtema 2.2. Requisitos funcionales y no funcionales. Propiedades emergentes. Requisitos cuantificables

Requisitos funcionales (RF)

Definición

Requisito funcional (Requisito funcional). Enunciado que describe una *función, servicio o tarea* que el sistema debe ejecutar. Responde a la pregunta: *¿qué debe hacer el sistema?* [**Sommerville2016**, **Pohl2024**].

Sommerville [**Sommerville2016**] caracteriza los RF como «las declaraciones de los servicios que el sistema debe proveer, cómo debe reaccionar ante entradas particulares y cómo debe comportarse en situaciones específicas». Para mantener uniformidad en el ERS, conviene adoptar una plantilla y un identificador único; una plantilla muy usada en proyectos ecuatorianos es:

RF-NN: *El sistema deberá [acción] [objeto] [cuando condición] [con el fin de propósito].*

Ejemplo en Ecuador

Ejemplo: sistema de turnos en un consultorio médico de Cuenca.

- **RF-01**: El sistema deberá permitir al paciente reservar turno en línea seleccionando especialidad, médico, fecha y hora disponibles.
- **RF-02**: El sistema deberá enviar confirmación por correo electrónico y mensaje SMS al paciente al confirmar la reserva.
- **RF-03**: El sistema deberá impedir reservar más de un turno activo simultáneo por paciente para la misma especialidad.
- **RF-04**: El sistema deberá generar un reporte mensual de citas atendidas, agrupado por médico y por especialidad, exportable a PDF y XLSX.

Requisitos no funcionales (Requisito no funcional / Non-Functional Requirement (sinónimo en inglés de RNF) (NFR))

Los **requisitos no funcionales** (RNF) describen *cualidades, restricciones y atributos* del sistema. No describen una función específica sino propiedades transversales: cuán rápido, cuán seguro, cuán mantenible, cuán portable. Pese a su nombre, su impacto suele ser *tan determinante para el éxito* como el de los requisitos funcionales [**Glinz2007**, **Chung2000**, **Eckhardt2016**].

Definición

Requisito no funcional. Atributo o restricción de un sistema que no describe una función concreta sino una *propiedad transversal* del producto, del proceso de desarrollo o del entorno. Suele expresarse mediante *métricas y umbrales* que permiten su verificación [**Glinz2007**, **Pohl2024**, **ISO25010_2023**].

La taxonomía clásica los agrupa en tres familias [**Sommerville2016**]:

Tabla 1.5. *Las tres familias de requisitos no funcionales según [***Sommerville2016***].*

Familia	¿Qué cubre?	Ejemplos típicos
Requisitos de producto	Atributos que el sistema mismo debe exhibir.	Rendimiento, fiabilidad, usabilidad, seguridad.
Requisitos organizacionales	Políticas y procedimientos del cliente o desarrollador.	Estándar de codificación, plataforma, idioma de la UI.
Requisitos externos	Restricciones legales, éticas, regulatorias.	Cumplir con LOPDP, formatos del SRI, IESS.

Ejemplos de RNF en el contexto ecuatoriano.

- **RNF-01** (Rendimiento): «El sistema deberá responder a la consulta de matrícula en menos de 3 s para el percentil 95 de las solicitudes durante la semana pico de inscripciones.»
- **RNF-02** (Seguridad): «El sistema deberá cifrar las contraseñas con bcrypt, factor de costo ≥ 12, y nunca almacenarlas en texto plano.»
- **RNF-03** (Disponibilidad): «El sistema de banca móvil deberá presentar una disponibilidad mensual no menor al 99,8 %, equivalente a un máximo de 87,7 minutos de indisponibilidad

por mes.»

- **RNF-04** (Usabilidad): «Un usuario sin entrenamiento previo deberá completar el registro de un comprobante electrónico en menos de 5 minutos en su primer intento.»
- **RNF-05** (Cumplimiento legal): «El sistema deberá cumplir con los lineamientos de la LOPDP del Ecuador (Registro Oficial 459 del 26 de mayo de 2021)»[**Ecuador_LOPDP_2021**].

Propiedades emergentes

Definición

Propiedad emergente. Cualidad del sistema considerado como un *todo*, que solo aparece cuando todos los componentes interactúan en su entorno operativo. *No puede atribuirse a ningún componente aislado* [**Sommerville2016**].

Ignorar las propiedades emergentes es uno de los errores más serios y costosos de la IR, porque son justamente las que determinan si el sistema funciona en la realidad. Son ejemplos clásicos:

- **Fiabilidad global**: aunque cada componente sea fiable al 99 %, la fiabilidad del sistema total no es 99 %, sino el resultado de cómo se acoplan todos los componentes y sus modos de fallo.
- **Tiempo de respuesta extremo a extremo**: depende de la red, los servidores, las bases de datos y los servicios de terceros (SRI, IESS, Banco Central). Optimizar solo el frontend no garantiza el resultado.
- **Seguridad**: ningún componente «es seguro»por sí solo. La seguridad emerge de la combinación de autenticación, autorización, cifrado, auditoría, hardening y políticas.
- **Usabilidad**: emerge de la interacción entre la interfaz, la terminología, los tiempos de respuesta y la consistencia entre módulos.

Ejemplo en Ecuador

Caso: banca móvil ecuatoriana. Una aplicación de banca móvil de un banco ecuatoriano puede tener componentes individualmente óptimos: una app rápida, un backend potente, una integración eficiente con el switch transaccional. Sin embargo, su disponibilidad emergente puede caer al 95 % si los procesos de respaldo, los enlaces con el Banco Central y la operación humana de atención no se diseñan en conjunto. La propiedad emergente «disponibilidad real para el cliente»no se logra sumando atributos aislados, se diseña pensando el sistema como un todo [**Charette2005**].

Requisitos cuantificables

Un requisito es **cuantificable** cuando contiene una métrica objetiva que permite verificarlo sin ambigüedad. Tanto Pohl como SWEBOK v4 insisten en que todo RNF debe ser, en la medida de lo posible, cuantificable; de lo contrario es solo un deseo [**Pohl2024**, **SWEBOK_v4_2024**]. La transformación de un enunciado vago en uno cuantificable suele requerir tres elementos:

1. Una **métrica** concreta (segundos, porcentaje, número de fallos por mes, transacciones por segundo, etc.).
2. Una **unidad clara y un punto de referencia** o instrumento de medición.

3. Una **condición de operación** (carga, contexto, percentil).

Tabla 1.6. *Conversión de enunciados vagos en requisitos cuantificables.*

Enunciado vago (incorrecto)	Enunciado cuantificable (correcto)
El sistema debe ser rápido.	El sistema deberá responder a una consulta de saldo en menos de 1,5 s para el 95 % de las solicitudes con 500 usuarios concurrentes.
La interfaz debe ser amigable.	Un usuario sin experiencia previa deberá poder completar la operación de transferencia en menos de 90 segundos en su primer intento.
Debe ser seguro.	El sistema deberá bloquear la cuenta tras 5 intentos fallidos de autenticación en una ventana de 10 minutos.
Debe estar siempre disponible.	El sistema deberá tener una disponibilidad mensual $\geq$ 99,8 %, medida sobre la ventana de servicio 24×7.
Debe ser fácil de mantener.	El equipo deberá poder corregir un defecto crítico (severidad 1) y desplegarlo en menos de 4 horas hábiles.

⚠ Advertencia / Error frecuente

Regla de oro. Si un requisito *no se puede medir*, no se puede verificar. Y si no se puede verificar, técnicamente no es un requisito: es un *deseo*. Antes de aprobar el ERS, el analista debe preguntarse por cada RNF: ¿qué prueba lo demostraría?, ¿con qué número se compararía el resultado? [**Mavin2009**, **Wiegers2013**].

1.2.3 Subtema 2.3. Requisitos del sistema vs. requisitos del software. Restricciones y dependencias

Niveles de requisitos: una jerarquía indispensable

La norma [**IEEE29148__2018**] distingue, dentro de un mismo proyecto, varios niveles de requisitos que se construyen unos a partir de otros y se trazan entre sí. No reconocer estos niveles es causa habitual de confusión entre clientes, analistas y desarrolladores. La tabla 1.7 resume los cuatro niveles canónicos.

Tabla 1.7. *Niveles canónicos de requisitos según IEEE 29148:2018* [**IEEE29148_2018**].

Nivel	Documento	Pregunta	Audiencia
Negocio / Misión	Business Reqs. – ConOps	¿Por qué se construye?	Alta dirección.
Stakeholder	StRS	¿Qué necesitan los stakeholders?	Usuarios, gerentes.
Sistema	SyRS	¿Qué hace el sistema completo?	Arquitectos.
Software	SRS / ERS	¿Qué hace el software?	Desarrolladores.

La figura 1.4 representa esta jerarquía como una pirámide invertida: en la cumbre se encuentran las metas de negocio (las más abstractas) y, descendiendo, los requisitos cada vez más detallados y asignables a componentes concretos.

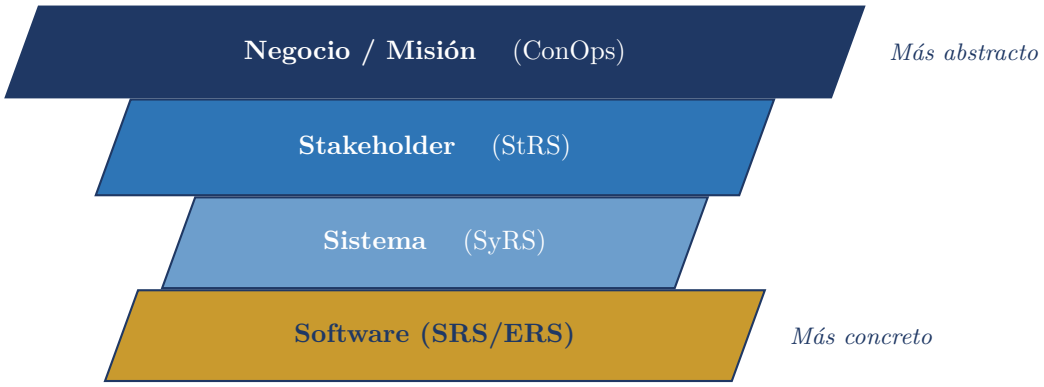


Figura 1.4. *Pirámide invertida de niveles de requisitos. Cada nivel se deriva del anterior mediante un proceso de refinamiento y asignación (allocation)* [**IEEE29148_2018**].

Requisitos del sistema

Los requisitos del sistema describen el comportamiento del sistema completo, entendido como la combinación de *hardware, software, personas, datos, instalaciones, materiales y procedimientos* [**IEEE29148_2018**, **Pohl2024**]. Un sistema, en este sentido amplio, no es solo un programa: es una solución completa que incluye al usuario, al ambiente y a los procesos organizacionales.

Ejemplo en Ecuador

Caso: pagos del IESS para empleadores. Como sistema, no es solo un portal web. Incluye: el portal del afiliado (software), los servidores en el datacenter (hardware), los integradores con bancos privados y el Banco del Pacífico (subsistemas externos), los lineamientos legales del Reglamento del IESS (procedimientos) y los empleadores que deben cargar planillas (personas). Si solo se especifica el software, se ignoran restricciones críticas como horarios de *cutoff* bancario, requerimientos de respaldo del datacenter o capacitación al empleador.

Requisitos del software

Los requisitos del software, documentados en el SRS o ERS en español, son una refinación de los requisitos del sistema *asignados al componente de software*. Describen únicamente lo que el software hace, ignorando explícitamente al hardware, al usuario y al entorno externo [**IEEE29148_2018**, **Sommerville2016**].

Cómo se derivan unos de otros. La derivación de requisitos del sistema a requisitos del software se realiza mediante un proceso de *asignación* o *allocation*: se decide qué porción de cada requisito del sistema será cubierta por software y qué porción por hardware, personas o procesos [**IEEE29148_2018**]. Cada decisión de asignación debe quedar trazada.

Tabla 1.8. *Asignación (allocation) de requisitos del sistema al software, con ejemplos ecuatorianos.*

Requisito del sistema	Asignación a software	Asignación a HW / personas
Pesar carga de banano para exportación con precisión ±50 g.	El módulo deberá leer y registrar el peso recibido del puerto serial.	Balanza certificada por la Agencia de Regulación; operario calibrado.
Facturar electrónicamente al SRI.	El software deberá generar el XML, firmarlo y transmitirlo al SRI [**SRI_FE_2024**].	Token criptográfico o HSM con certificado digital; contador con clave.
Permitir cobros sin conexión en zonas rurales.	El software deberá almacenar transacciones en cola local y sincronizar al volver la conexión.	Dispositivo POS con batería ≥ 8 h; liquidación diaria.

Restricciones

Definición

Restricción. Condición externa al equipo de desarrollo que *limita las opciones de diseño y de implementación* del sistema. A diferencia de un requisito –que el equipo intenta cumplir mediante una solución–, una restricción *no se negocia*: viene impuesta por el contexto [**Pohl2024**, **IEEE29148_2018**].

Reconocer las restricciones a tiempo es vital, porque cambian el espacio de soluciones posibles. La tabla 1.9 clasifica los tipos de restricción más comunes con ejemplos de Ecuador.

Tabla 1.9. *Tipos comunes de restricciones, con ejemplos en Ecuador.*

Tipo	Descripción	Ejemplo en Ecuador
Tecnológicas	Tecnologías obligatorias o prohibidas.	El portal del Ministerio debe correr sobre PostgreSQL y Linux por política de soberanía tecnológica.
Legales / Normativas	Leyes, reglamentos o normas obligatorias.	Cumplir con la LOPDP [**Ecuador_LOPDP_2021**] y los esquemas del SRI [**SRI_FE_2024**].
Económicas	Limitaciones de presupuesto o costos.	El proyecto cuenta con USD 45 000 para todo el ciclo de vida del primer año.
Temporales	Plazos de entrega impuestos.	Debe estar operativo antes del 1 de enero por entrada en vigor de una resolución.
De interfaz	Sistemas con los que debe convivir.	Debe integrarse con el ESIGEF (Min. Finanzas) y con el portal del SERCOP.
Organizacionales	Políticas internas, estructura o cultura.	Debe usar el directorio activo institucional para la autenticación.
Físicas / Ambientales	Condiciones del entorno operativo.	El POS debe operar a 35 °C y 90 % de humedad en zonas costeras.

Dependencias entre requisitos

Definición

Dependencia. Relación entre dos requisitos en la que el cumplimiento, la prioridad o el comportamiento de uno depende del otro. Detectar dependencias evita inconsistencias y guía la planificación de *releases* [**SWEBOK_v4_2024**, **Pohl2024**].

SWEBOK v4 las clasifica en cinco categorías [**SWEBOK_v4_2024**], que se sintetizan en la tabla 1.10.

Tabla 1.10. *Tipos de dependencias entre requisitos.*

Tipo	Significado	Ejemplo (matrícula)
Requiere (Requires)	RF-A no tiene sentido si no existe RF-B.	**RF-12** «pagar matrícula»requiere **RF-08** «generar orden de pago».
Conflicto (Conflicts)	Dos requisitos no pueden cumplirse simultáneamente.	Maximizar disponibilidad 24×7 vs. ventanas de mantenimiento programado de 4 h.
Refina (Refines)	Un requisito detalla a otro de mayor nivel.	**RF-08.1** «calcular descuento de beca»refina a **RF-08** «generar orden de pago».
Implica (Implies)	Cumplir RF-A obliga a cumplir RF-B aunque no se mencione.	Aceptar pagos con tarjeta implica cumplir PCI-DSS.
Excluye (Excludes)	RF-A solo se cumple si RF-B no se selecciona.	Pago en efectivo en taquilla excluye recargo por uso de pasarela.

Ejemplo en Ecuador

Caso: dependencia mal gestionada en un proyecto público. En un sistema regional de salud, el equipo aprobó **RF-23** «el sistema deberá enviar receta electrónica al farmacéutico»sin notar que dependía de **RF-09** «el sistema deberá identificar inequívocamente al farmacéutico colegiado». **RF-09** fue desestimado por «menos prioritario». El resultado: en producción, las recetas llegaban a destinatarios genéricos, sin trazabilidad legal, exponiendo a la institución a responsabilidades sanitarias. Una matriz de dependencias, elaborada en la fase de IR, habría evitado el problema.

1.2.4 Subtema 2.4. Taxonomía de requisitos de calidad (ISO/IEC 25010:2023). Los llamados NFR

La importancia de un modelo de calidad estandarizado

ISO/IEC 25010 define un *modelo de calidad* como un marco que descompone la calidad del producto software en **características y subcaracterísticas medibles** [**ISO25010_2023**]. Esta norma reemplazó al ISO/IEC 9126 y es hoy la referencia internacional más usada para estructurar los requisitos no funcionales. Como recuerda SWEBOK v4 [**SWEBOK_v4_2024**], los NFR no se inventan al azar: se derivan de un modelo de calidad.

Novedad: ISO/IEC 25010:2023 (segunda edición)

En noviembre de 2023, ISO publicó la **segunda edición** del estándar, con cambios sustantivos respecto de la versión 2011 que conviene conocer [**ISO25010_2023**]:

- Se incorpora **Safety** (seguridad funcional o operacional) como novena característica de calidad. La versión 2011 solo tenía ocho. Esta es la novedad más importante.
- *Usability* fue **renombrada como Interaction capability**. El cambio refleja que el modelo

se aplica ahora a sistemas con múltiples modos de interacción (voz, gestos, IoT) y no solo a interfaces gráficas.

- *Portability* fue **renombrada como Flexibility** y se le añadió la subcaracterística *scalability*.
- Se añadieron las subcaracterísticas *inclusivity* (en Interaction capability), *resistance* (en Security) y *user engagement* y *faultlessness* (que reemplazan a *aesthetics* y *maturity*).

La tabla 1.11 resume las principales diferencias entre ambas ediciones.

Tabla 1.11. *Cambios principales entre ISO/IEC 25010:2011 e ISO/IEC 25010:2023 [***ISO25010_2011**, **ISO25010_2023***].*

Aspecto	Edición 2011	Edición 2023
N.º de características	8	9 (se añade Safety)
Usability	Sí (presente)	Renombrada como *Interaction capability*
Portability	Sí (presente)	Renombrada como *Flexibility*
Subcaracterísticas nuevas	—	inclusivity, resistance, scalability, faultlessness
Foco	Productos de software	Productos de software *y otros productos TIC*

Las nueve características de calidad de ISO/IEC 25010:2023

La figura 1.5 resume gráficamente las nueve características y la tabla 1.12 las describe con sus subcaracterísticas.

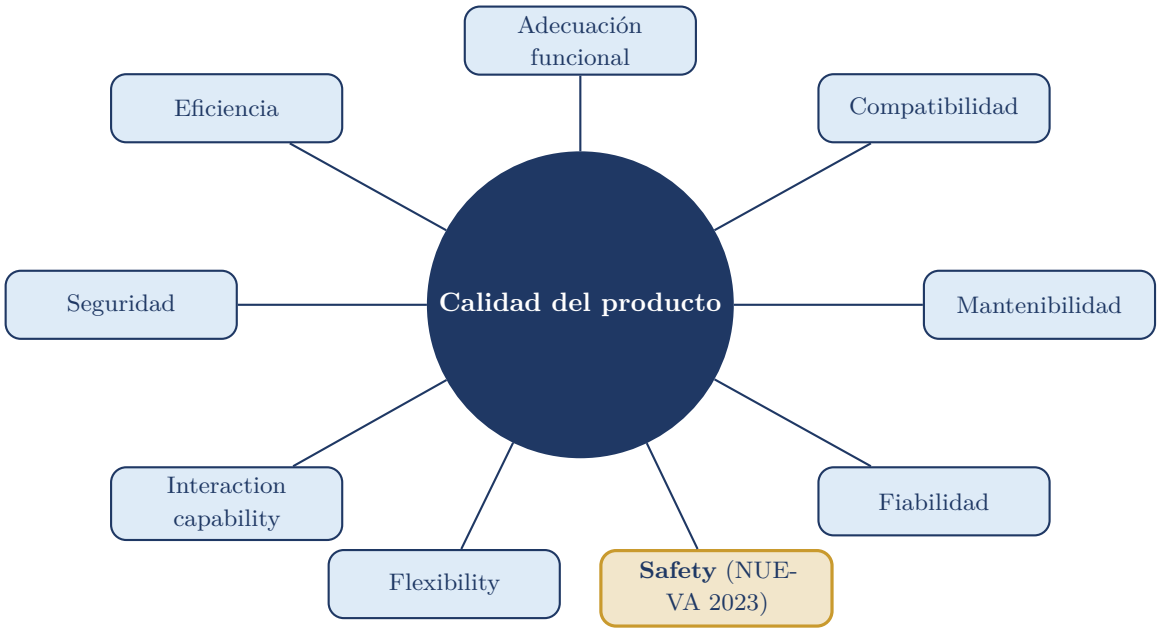


Figura 1.5. *Las nueve características de calidad de ISO/IEC 25010:2023 [***ISO25010_2023***]. La novena característica, **Safety**, constituye la novedad principal de la edición 2023.*

Tabla 1.12. *Las nueve características de calidad ISO/IEC 25010:2023 con sus subcaracterísticas* [**ISO25010_2023**].

#	Característica	Idea central	Subcaracterísticas
1	Adecuación funcional	Hace lo que debe hacer.	Completitud, corrección, pertinencia funcional.
2	Eficiencia de desempeño	Lo hace usando recursos razonables.	Comportamiento temporal, utilización de recursos, capacidad.
3	Compatibilidad	Convive con otros sistemas.	Coexistencia, interoperabilidad.
4	Interaction capability	Es fácil y satisfactorio de usar (antes Usability).	Reconocibilidad, aprendizaje, operabilidad, protección frente a errores, *user engagement*, accesibilidad, *inclusivity*, autodescriptividad.
5	Fiabilidad	Funciona sin fallar bajo condiciones acordadas.	*Faultlessness*, disponibilidad, tolerancia a fallos, recuperabilidad.
6	Seguridad	Protege información y servicios.	Confidencialidad, integridad, no repudio, responsabilidad, autenticidad, *resistance*.
7	Mantenibilidad	Se modifica con bajo costo.	Modularidad, reusabilidad, analizabilidad, modificabilidad, capacidad de prueba.
8	Flexibility	Se adapta a otros entornos (antes Portabilidad).	Adaptabilidad, instalabilidad, reemplazabilidad, *scalability*.
9	**Safety** (nueva)	No causa daño a personas, propiedad o entorno.	Restricción operacional, identificación de riesgos, *fail safe*, alerta de peligro, integración segura.

Detalle de cada característica con ejemplos ecuatorianos

1. Adecuación funcional. Mide si el software hace lo que debe hacer y lo hace correctamente. Sus subcaracterísticas son completitud, corrección y pertinencia funcional [**ISO25010_2023**].

Ejemplo en Ecuador

Facturación electrónica. Un sistema contable ecuatoriano debe generar comprobantes con todos los campos exigidos por el SRI (completitud), debe calcular correctamente el IVA al porcentaje vigente y aplicar bien las retenciones (corrección), y debe permitir al contador realizar las tareas usuales de cierre mensual sin pasos innecesarios (pertinencia) [**SRI_FE_2024**].

2. Eficiencia de desempeño. Cantidad de recursos consumidos y tiempos de respuesta entregados, bajo condiciones específicas. Subcaracterísticas: comportamiento temporal, utilización de recursos y capacidad.

Ejemplo en Ecuador

Plataforma de matrícula UTEQ. «El sistema deberá soportar 1 500 inscripciones concurrentes con un tiempo de respuesta promedio menor a 3 s y consumo de CPU por instancia inferior al 75 % durante la semana pico». Aquí se cuantifican las tres subcaracterísticas: tiempo, recursos y capacidad.

3. Compatibilidad. Capacidad para coexistir con otros productos y para interoperar con ellos. Subcaracterísticas: coexistencia (compartir recursos sin afectar a otros) e interoperabilidad (intercambiar y usar información con otros sistemas).

Ejemplo en Ecuador

Banca móvil. Una solución de banca móvil ecuatoriana debe interoperar con el Banco Central (transferencias SPI), con BANRED para tarjetas, con el SRI para retenciones y con cooperativas ecuatorianas, sin afectar el rendimiento de los servicios de los demás.

4. Interaction capability (antes Usability). Grado en que el software puede ser usado por usuarios específicos para alcanzar metas con efectividad, eficiencia y satisfacción en un contexto de uso. La edición 2023 añadió *inclusivity* y *user engagement* como subcaracterísticas [**ISO25010_2023**].

Ejemplo en Ecuador

App municipal en zonas rurales. Una app del GAD Municipal para reportar baches o incidentes urbanos en zonas rurales de Los Ríos debe ser comprensible para un usuario con baja exposición tecnológica (reconocibilidad), enseñar su uso en pocos minutos (aprendizaje), resistir errores como subir fotos demasiado grandes (protección frente a errores), cumplir con pautas WCAG 2.1 AA (accesibilidad e inclusivity) y contener guías contextuales (autodescriptividad).

5. Fiabilidad. Capacidad para mantener un nivel especificado de desempeño bajo condiciones acordadas. Subcaracterísticas: *faultlessness*, disponibilidad, tolerancia a fallos y recuperabilidad.

Ejemplo en Ecuador

Sistema de turnos del MSP. «El sistema deberá tener una disponibilidad mensual $\geq 99{,}9\,\%$ (disponibilidad), tolerar la caída de un nodo sin interrumpir el servicio (tolerancia a fallos) y recuperarse en menos de 15 minutos tras una falla mayor (recuperabilidad)».

6. Seguridad. Capacidad de proteger información y datos. Subcaracterísticas: confidencialidad, integridad, no repudio, responsabilidad, autenticidad y *resistance* (nueva en 2023).

Ejemplo en Ecuador

Historia clínica electrónica. Un sistema de historia clínica de un hospital del IESS debe asegurar que solo el médico tratante acceda al diagnóstico (confidencialidad), evitar adulteraciones (integridad), guardar evidencia firmada de la prescripción (no repudio), registrar cada acceso (responsabilidad), autenticar con doble factor (autenticidad) y resistir intentos de inyección SQL (*resistance*) [**Ecuador_LOPDP_2021**].

7. Mantenibilidad. Eficiencia con que se puede modificar el software. Subcaracterísticas: modularidad, reusabilidad, analizabilidad, modificabilidad y capacidad de prueba.

Ejemplo en Ecuador

Sistema académico universitario. Cuando la SENESCYT cambia las reglas de homologación de asignaturas, el sistema académico de una universidad ecuatoriana debe poder modificar el módulo afectado sin tocar los demás (modularidad y modificabilidad), permitir analizar el impacto del cambio (analizabilidad) y verificarlo con pruebas automatizadas (capacidad de prueba).

8. Flexibility (antes Portabilidad). Efectividad con que el software puede transferirse de un entorno a otro o escalar en el actual. Subcaracterísticas: adaptabilidad, instalabilidad, reemplazabilidad y *scalability* (nueva en 2023).

Ejemplo en Ecuador

Migración a la nube gubernamental. Un sistema desarrollado para servidores on-premise del Ministerio puede migrarse a la nube gubernamental del Ecuador. Si fue diseñado pensando en flexibilidad, puede ejecutarse sobre Linux y Docker (adaptabilidad), instalarse con un script reproducible (instalabilidad), reemplazar al sistema legado sin pérdida de datos (reemplazabilidad) y escalar horizontalmente al crecer la carga (*scalability*).

9. Safety (NUEVA en 2023). Capacidad del software de no provocar daño a las personas, a la propiedad o al entorno. Subcaracterísticas: restricción operacional, identificación de riesgos, *fail safe*, alerta de peligro e integración segura [**ISO25010_2023**].

Ejemplo en Ecuador

Sistema agroindustrial automatizado. En una empacadora bananera de Quevedo, el software que controla las cintas y elevadores debe detenerse automáticamente si detecta una condición anormal (*fail safe*), advertir al operario sobre obstrucciones (alerta de peligro), respetar los límites de velocidad de cintas (restricción operacional) e integrarse de forma segura con sensores y PLC industriales (integración segura).

Cómo escribir un NFR usando ISO/IEC 25010:2023

La taxonomía no es un fin en sí misma: es una guía de redacción. Una buena receta práctica para el estudiante, alineada con [**Pohl2024**] y [**Wiegers2013**], es la siguiente:

1. **Identificar** el atributo de calidad (por ejemplo, Eficiencia → Comportamiento temporal).
2. **Definir** un escenario operativo (carga, contexto, percentil).

3. **Indicar** la métrica y el valor objetivo.
4. **Indicar** el método de verificación.

RNF-XX *[Eficiencia – Comportamiento temporal]*: «Bajo una carga concurrente de 800 usuarios, el sistema deberá responder a la consulta de saldo en menos de 1,5 s para el percentil 95 de las solicitudes. Verificación: prueba de carga con JMeter durante 30 minutos.»

1.3 Mapa síntesis de la Semana 1

Cerramos la semana con un mapa que conecta los cuatro grandes pilares vistos. La idea es que el estudiante visualice cómo encajan entre sí cuando aborde su primer ERS.

Tabla 1.13. *Mapa de cierre de la Semana 1.*

Pilar	Aporte conceptual	Documento donde aparece
Definición y tipos (2.1)	Vocabulario base; producto vs. proceso.	Glosario y sección de tipos del ERS.
Funcionales y NF (2.2)	Diferenciar el qué del cómo de bien; cuantificar.	Secciones 3 y 4 del ERS (IEEE 29148).
Sistema vs. software (2.3)	Niveles, restricciones, dependencias.	Diagrama de contexto y matriz de trazabilidad.
ISO/IEC 25010:2023 (2.4)	Taxonomía estructurada para los NFR.	Sección de requisitos de calidad del ERS.

1.4 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 1 (en clase, 2 horas).

Objetivo: aplicar la plantilla de redacción y las nueve características de IEEE 29148 a un caso real ecuatoriano.

Caso: la UTEQ desea construir un sistema en línea de préstamo de libros para sus 17 carreras.

Tareas en equipos de 3–4 estudiantes:

1. Identificar al menos cinco stakeholders distintos.
2. Redactar diez requisitos funcionales con la plantilla **RF-NN** y verificar cada uno contra las nueve características de IEEE 29148 [**IEEE29148_2018**].
3. Identificar tres requisitos vagos suministrados por el docente y reescribirlos como requisitos cuantificables.
4. Discutir en plenaria los hallazgos.

Práctica / Actividad

Tarea autónoma n.º 1 (4 horas, individual).

Caso: un GAD cantonal del Ecuador desea contratar el desarrollo de un sistema de cobro de impuestos prediales en línea, integrado con el portal del SRI y con un proveedor de pasarela de pagos local.

Tareas:

1. Identificar al menos tres stakeholders y sus necesidades preliminares.
2. Redactar cinco RF y cinco RNF, siguiendo la plantilla y las características de [**IEEE29148_2018**].
3. Clasificar cada RNF según las **nueve** características de ISO/IEC 25010:2023 [**ISO25010_2023**].
4. Identificar tres restricciones y dos dependencias entre requisitos, justificando cada una.
5. Para cada RNF, indicar la métrica y el método de verificación.

Entregable: documento PDF de máximo 6 páginas. La rúbrica ponderará especialmente la verificabilidad y la cuantificación de los RNF.

Práctica / Actividad

Lectura previa para la Semana 2 (aula invertida).

- Pohl, K. *Requirements Engineering: Fundamentals, Principles, and Techniques*, 2.ª ed., capítulos 1–3 [**Pohl2024**].
- IEEE/ISO/IEC 29148:2018, cláusulas 5 y 6 [**IEEE29148_2018**].
- Daun et al. (2023), *A systematic literature review of requirements engineering education*, Requirements Engineering Journal [**Daun2023**].

1.5 Resumen ejecutivo de la semana

- La IR es la actividad sobre la cual se sostiene toda la construcción posterior del software. Errores aquí cuestan hasta cien veces más en producción [**Boehm2001**].
- Un **requisito** es, según IEEE 29148:2018, un enunciado de una necesidad y sus restricciones; una capacidad observable que debe poseer el sistema; o su representación documentada [**IEEE29148_2018**].
- Los **requisitos de producto** describen el sistema; los **de proceso** describen cómo se construye [**SWEBOK_v4_2024**].
- Los **requisitos funcionales** dicen qué hace el sistema; los **no funcionales** describen sus cualidades transversales [**Sommerville2016**, **Glinz2007**].
- Las **propiedades emergentes** solo aparecen cuando todos los componentes interactúan; no se pueden atribuir a un componente aislado [**Sommerville2016**].
- Todo RNF debe ser **cuantificable**: si no se puede medir, no se puede verificar [**Pohl2024**].
- Existen cuatro **niveles** de requisitos (negocio, stakeholder, sistema, software) que se trazan entre sí [**IEEE29148_2018**].

- Las **restricciones** no se negocian; las **dependencias** deben mapearse explícitamente para evitar inconsistencias [**Pohl2024**, **SWEBOK_v4_2024**].
- ISO/IEC 25010:2023 introduce **nueve características de calidad** (frente a las ocho de la versión 2011), añadiendo **Safety** y renombrando Usabilidad y Portabilidad como *Interaction capability* y *Flexibility* respectivamente [**ISO25010_2023**].